CENTRO UNIVERSITÁRIO PARAÍSO

INTELIGÊNCIA ARTIFICIAL

TDE2 DA DISCIPLINA:

Trabalho — Agentes Inteligentes em Games

Arthur Rodrigues de Carvalho - 20241130037

Ramon Porfírio Macedo - 20241130023

Thalys Eduardo Ferreira Cazuza - 20241180148

AGENTE INTELIGENTE “MOTOBOY RIVAL” PARA O JOGO DELIVERY RUSH

Relatório técnico: proposta de Inteligência Artificial aplicada a NPCs competidores

Professor(a): Alexsandro Oliveira Alexandrino

JUAZEIRO DO NORTE-CE

2026

SUMÁRIO

1 APRESENTAÇÃO DO JOGO..... 3
2 CONTEXTUALIZAÇÃO DO PROBLEMA.....5
3 JUSTIFICATIVA PARA A UTILIZAÇÃO DE INTELIGÊNCIA ARTIFICIAL..... 6
4 PROBLEMA E PROPOSTA DE SOLUÇÃO..... 7
4.1 Descrição do comportamento a ser solucionado.....7
4.2 Funcionamento esperado do agente..... 7
4.3 Justificativa das técnicas de IA escolhidas..... 8
5 FUNDAMENTAÇÃO TEÓRICA.....9
5.1 Agentes inteligentes.....9
5.2 Máquinas de estados finitos..... 10
5.3 Árvores de decisão..... 10
5.4 Busca A*..... 10
5.5 Sistemas baseados em regras e ajuste dinâmico de dificuldade..... 11
6 APLICAÇÃO AO PROJETO DE GAMES.....12
6.1 Integração do agente ao jogo..... 12
6.2 Entradas utilizadas pelo agente.....12
6.3 Decisões e ações produzidas pelo agente..... 13
6.4 Interação entre o agente e os demais elementos do jogo.....13
6.5 Ajuste dinâmico de dificuldade..... 14
7 MODELAGEM DA SOLUÇÃO..... 15
7.1 Arquitetura do agente..... 15
7.2 Máquina de estados.....15
7.3 Árvore de decisão para avaliação de pedidos..... 17
7.4 Planejamento de rotas com A*..... 18
7.5 Decisão de cortar caminho pela calçada.....20
7.6 Pseudocódigo do agente..... 21
7.7 Exemplos de situações de entrada e decisões esperadas.....21
8 CONSIDERAÇÕES FINAIS..... 23
REFERÊNCIAS..... 24

1 APRESENTAÇÃO DO JOGO

Delivery Rush é um jogo arcade desenvolvido na plataforma Roblox, programado na linguagem Luau, no qual o jogador assume o papel de um entregador de moto em uma grande cidade dividida em cinco bairros: Centro, Bairro Residencial, Zona Comercial, Bairro Nobre e Centro Financeiro. A proposta é ser rápido, divertido e “viciante”, sem simulação realista: a moto não capota e as colisões apenas reduzem a velocidade ou empurram levemente o veículo.

O Delivery Rush é o projeto de PI em Games de parte dos integrantes desta equipe. Como a equipe reúne alunos de grupos de PI diferentes, o jogo foi escolhido em comum acordo como estudo de caso, e a proposta do agente inteligente apresentada neste relatório foi elaborada por todos os integrantes.

O ciclo principal de jogo (game loop) consiste em: receber um pedido, ir até o restaurante indicado, coletar o pedido, levá-lo até o cliente, receber o pagamento e investir o dinheiro em upgrades. Existem seis níveis de moto (do nível 0 ao 5), cada um mais rápido que o anterior e com um visual novo e mais esportivo, além de bairros que precisam ser desbloqueados e que pagam mais pelas entregas.

O pagamento é calculado no servidor e recompensa a velocidade: valor final = pagamento base do bairro × multiplicador de velocidade × gorjeta (±10%). Quem entrega em até 50% do tempo ideal recebe 2,5×; no tempo ideal, 1,0×; e a partir de 200% do tempo, 0,4× (nunca zero). A cidade foi gerada de forma procedural, com quarteirões de tamanhos diferentes, becos e superquarteirões, e está repleta de desafios: trânsito com mais de dois mil veículos, espinhos que furam o pneu (a moto fica 3 segundos parada e o jogador paga o conserto), manchas de óleo, buracos, lombadas, lama, hidrantes estourados, radares de velocidade, obras, desvios, rampas de salto e bolas de demolição. O jogador também pode cortar caminho pela calçada, mas fica mais lento e paga uma taxa que cresce a cada 1,6 segundo, e atropelar pedestres gera multa.

Quadro 1 – Ficha técnica do jogo Delivery Rush

Item	Descrição
Gênero	Arcade / corrida / gerenciamento de entregas
Plataforma e linguagem	Roblox (PC, celular e tablet) — Luau

Item	Descrição
Modo de jogo	Multijogador online, cada jogador com sua própria moto e seus pedidos
Mecânicas principais	Pilotagem arcade, manobra de grau e turbo, saltos em rampas, pedidos com tempo ideal e pagamento por velocidade, upgrades de moto, desbloqueio de bairros
Desafios do ambiente	Trânsito, pedestres, espinhos, óleo, buracos, lombadas, lama, radares, obras, bolas de demolição e taxa progressiva por andar fora da rua
Arquitetura técnica	Cliente–servidor: o servidor calcula dinheiro, valida entregas e salva os dados (DataStore); o cliente cuida da pilotagem, da câmera e dos efeitos visuais

Fonte: elaborado pela equipe (2026).

2 CONTEXTUALIZAÇÃO DO PROBLEMA

Na versão atual de Delivery Rush, todos os personagens não jogáveis (NPCs) são **reativos e roteirizados**. Os carros, ônibus e motoboys do trânsito percorrem rotas fixas em loop, indo e voltando pelas mesmas avenidas, e a única “decisão” que tomam é frear quando a moto do jogador está logo à frente. Os pedestres caminham em volta dos quarteirões em trajetórias pré-definidas. Nenhum NPC persegue objetivos, avalia opções ou se adapta ao que acontece na partida.

Isso gera alguns problemas de experiência:

- **Falta de competição:** o tema do jogo é o de aplicativos de entrega, em que muitos entregadores disputam os mesmos pedidos, mas o jogador nunca sente essa pressão; os pedidos estão sempre à disposição dele.
- **Cidade com aparência “decorativa”:** os demais motoboys apenas circulam e nunca fazem entregas de verdade, o que quebra a imersão.
- **Servidores vazios:** quando há poucos jogadores online, a experiência se torna solitária e sem referência de desempenho.
- **Desafio previsível:** a dificuldade depende apenas de obstáculos estáticos; depois de algumas horas, o jogador decora o mapa e o jogo perde a tensão.

O problema abordado neste relatório é, portanto: **como tornar o mundo de Delivery Rush mais dinâmico e competitivo, com NPCs que realmente participem da economia de entregas, sem tornar o jogo injusto ou frustrante para o jogador?**

3 JUSTIFICATIVA PARA A UTILIZAÇÃO DE INTELIGÊNCIA ARTIFICIAL

Um entregador rival não pode ser resolvido com um roteiro fixo. Ele precisa escolher entre pedidos que mudam o tempo todo, navegar por uma cidade irregular (com trechos removidos, becos e obras que bloqueiam ruas), reagir a obstáculos e ao trânsito e, principalmente, competir de forma equilibrada com um jogador humano, cujo desempenho varia bastante. Esse tipo de comportamento, orientado a objetivos e sensível ao contexto, é justamente o campo das técnicas de Inteligência Artificial (IA) para jogos (MILLINGTON; FUNGE, 2009).

A IA permite criar um agente que **percebe** o estado do jogo, **decide** o que fazer e **age** de forma crível, além de ajustar a própria dificuldade para manter o jogador em um estado de desafio adequado — nem fácil demais, nem impossível —, conceito conhecido como ajuste dinâmico de dificuldade (HUNICKE, 2005) e relacionado à teoria do fluxo (CSIKSZENTMIHALYI, 1990). As técnicas propostas (máquina de estados, árvore de decisão, busca A* e regras) são leves do ponto de vista computacional e podem ser executadas no próprio servidor do Roblox, junto com os sistemas já existentes.

4 PROBLEMA E PROPOSTA DE SOLUÇÃO

4.1 Descrição do comportamento a ser solucionado

Deseja-se incluir no jogo o **Motoboy Rival**: um entregador controlado por IA, com a mesma moto e as mesmas regras do jogador, que disputa os pedidos da cidade. Para que a proposta funcione, o agente precisa atender aos seguintes requisitos:

1. **Escolher pedidos rentáveis:** avaliar os pedidos livres e aceitar apenas os que valem a pena, considerando distância, pagamento e a posição do jogador.
2. **Encontrar a rota mais rápida:** planejar o caminho pela malha de ruas, levando em conta o custo dos obstáculos e do trânsito.
3. **Replanejar:** mudar a rota quando surgir uma obra, um espinho ou um congestionamento no caminho.
4. **Respeitar as regras:** não atropelar pedestres, evitar a calçada (salvo quando realmente compensa), reduzir a velocidade nos radares e sofrer os mesmos efeitos dos obstáculos.
5. **Recuperar-se de falhas:** se ficar preso ou tiver o pneu furado, voltar ao comportamento normal sozinho.
6. **Ser justo:** adaptar agressividade, tempo de reação e velocidade ao nível e ao desempenho do jogador.

4.2 Funcionamento esperado do agente

O Motoboy Rival começa **ocioso**, circulando pelo bairro. Quando um pedido aparece no mural, ele passa a **avaliá-lo** com uma árvore de decisão. Se decidir aceitar, calcula com o algoritmo A* a rota de menor custo até o restaurante e passa a segui-la, pilotando de forma parecida com o jogador: acelera nas retas, freia atrás de carros, troca de faixa para desviar de espinhos e pode usar o turbo em trechos longos. Ao chegar, **coleta** o pedido, calcula uma nova rota até o cliente e o **entrega**, voltando ao estado ocioso.

Se o jogador aceitar primeiro o mesmo pedido, o rival desiste e volta a ficar ocioso. Por outro lado, se um pedido ficar muito tempo sem ser aceito pelo jogador,

o rival pode pegá-lo, e o jogador vê a mensagem “Um motoboy rival pegou este pedido!”. Isso cria uma pressão de tempo natural e faz a cidade parecer viva. Os rivais aparecem no mundo como NPCs com nome e cor próprios e disputam o placar da rodada com os jogadores; eles não recebem dinheiro real, apenas pontuação.

4.3 Justificativa das técnicas de IA escolhidas

A solução combina quatro técnicas clássicas de IA para jogos, cada uma responsável por um nível de decisão. Essa divisão de responsabilidades é comum na indústria, pois cada técnica é simples de implementar, depurar e balancear (BUCKLAND, 2005; MILLINGTON; FUNGE, 2009).

Quadro 2 – Técnicas escolhidas e justificativa

Técnica	Responsabilidade no agente	Por que foi escolhida
Máquina de estados finitos (FSM)	Define o modo de comportamento atual (ocioso, indo ao restaurante, entregando, etc.)	O trabalho de um entregador tem etapas bem definidas e sequenciais; a FSM deixa o comportamento previsível, fácil de visualizar e de depurar
Árvore de decisão	Escolhas táticas: aceitar ou recusar um pedido e como reagir a obstáculos	É interpretável (o designer entende cada decisão), fácil de ajustar e pode, no futuro, ser aprendida a partir de dados
Busca A*	Planejar a rota de menor custo na malha de ruas	É completa e ótima quando a heurística é admissível e muito eficiente em grafos do tamanho da cidade do jogo
Sistema de regras (ajuste dinâmico de dificuldade)	Regular agressividade, reação e velocidade do rival	Regras SE–ENTÃO permitem que o designer controle o equilíbrio do jogo de forma transparente

Fonte: elaborado pela equipe (2026).

Técnicas baseadas em aprendizado, como redes neurais e aprendizado por reforço, foram consideradas, mas não foram escolhidas como núcleo da solução: exigem grande volume de treino em simulação, produzem comportamentos difíceis de explicar e de balancear (“caixa-preta”) e são mais pesadas para o ambiente Luau do Roblox. Elas aparecem na seção 8 como trabalho futuro, para ajustar automaticamente os parâmetros do agente.

5 FUNDAMENTAÇÃO TEÓRICA

5.1 Agentes inteligentes

Segundo Russell e Norvig (2022), um **agente** é qualquer entidade que percebe o seu ambiente por meio de sensores e age sobre ele por meio de atuadores. O comportamento do agente é descrito por uma função que mapeia a sequência de percepções em ações. Os autores classificam os agentes em: reativos simples (agem só pela percepção atual), reativos baseados em modelo (mantêm um estado interno do mundo), baseados em objetivos (escolhem ações que levam a um objetivo) e baseados em utilidade (escolhem a ação de maior benefício esperado).

O Motoboy Rival é um **agente baseado em objetivos e em utilidade, com modelo interno do mundo**: ele guarda o mapa de ruas com custos, persegue o objetivo de concluir entregas e compara alternativas (pedidos, rotas e atalhos) pelo benefício que trazem. A especificação da tarefa pode ser descrita pelo modelo PEAS (desempenho, ambiente, atuadores e sensores), apresentado no Quadro 3.

Quadro 3 – Descrição PEAS do Motoboy Rival

Elemento	Descrição
Desempenho (P)	Entregas concluídas por minuto, pontuação da rodada, disputas vencidas, respeito às regras (sem atropelamentos e sem multas) e equilíbrio com o jogador
Ambiente (E)	Cidade de Delivery Rush: ruas e becos, pedidos, restaurantes e clientes, trânsito, pedestres, obstáculos e outros jogadores
Atuadores (A)	Acelerar, frear, virar, trocar de faixa, usar turbo, saltar rampas, aceitar, coletar e entregar pedidos
Sensores (S)	Posição e velocidade próprias, mural de pedidos, obstáculos no trecho à frente, veículos e pedestres próximos, posição e pedido atual do jogador

Fonte: elaborado pela equipe (2026).

Quanto às propriedades do ambiente, trata-se de um ambiente **parcialmente observável** (por justiça, o rival só “enxerga” até certa distância), **estocástico e dinâmico** (o trânsito e o jogador mudam o tempo todo), **sequencial**, **contínuo** e **multiagente competitivo**, pois o rival disputa recursos (os pedidos) com o jogador.

5.2 Máquinas de estados finitos

Uma máquina de estados finitos (FSM, do inglês finite state machine) pode ser definida formalmente como M = (Q, Σ, δ, q0), em que Q é um conjunto finito de estados, Σ é o conjunto de eventos de entrada, δ: Q × Σ → Q é a função de transição e q0 é o estado inicial. Em jogos, cada estado encapsula um comportamento (por exemplo, “patrulhar” ou “fugir”), com ações executadas ao entrar, a cada atualização e ao sair do estado; as transições são disparadas por condições do jogo (MILLINGTON; FUNGE, 2009).

A FSM é uma das técnicas mais usadas em IA de jogos por ser simples, eficiente e fácil de visualizar em diagramas. Sua principal limitação é o crescimento do número de transições em comportamentos muito complexos, o que pode ser contornado com máquinas de estados hierárquicas (HFSM), em que um estado contém uma sub-máquina.

5.3 Árvores de decisão

Uma árvore de decisão é uma estrutura em que cada nó interno testa um atributo da situação, cada ramo representa um resultado do teste e cada folha contém uma decisão ou ação (RUSSELL; NORVIG, 2022). Para decidir, o agente percorre a árvore da raiz até uma folha, respondendo às perguntas pelo caminho.

As árvores podem ser construídas manualmente pelo designer (caso desta proposta) ou aprendidas a partir de exemplos, com algoritmos como o ID3 (QUINLAN, 1986), que escolhe em cada nó o atributo de maior ganho de informação. A entropia de um conjunto S é $H(S) = -\Sigma\, p_i \cdot \log_2 p_i$, e o ganho de um atributo A é $\text{Ganho}(S, A) = H(S) - \Sigma\, (|S_v| / |S|) \cdot H(S_v)$. Essa possibilidade permite, no futuro, gerar a árvore a partir de registros de partidas de bons jogadores.

5.4 Busca A*

O A* (HART; NILSSON; RAPHAEL, 1968) é um algoritmo de busca informada que encontra o caminho de menor custo entre dois nós de um grafo. Ele expande primeiro o nó com menor valor de **f(n) = g(n) + h(n)**, em que g(n) é o custo real do início até n e h(n) é uma estimativa (heurística) do custo de n até o objetivo.

Se a heurística for **admissível** — isto é, nunca superestimar o custo real —, o A* garante encontrar o caminho ótimo; se, além disso, for **consistente**, cada nó precisa ser expandido no máximo uma vez. Com h(n) = 0, o A* se reduz ao algoritmo de Dijkstra, que também é ótimo, mas explora muito mais nós. Por isso o A* é o padrão da indústria para navegação de personagens em jogos (YANNAKAKIS; TOGELIUS, 2018).

5.5 Sistemas baseados em regras e ajuste dinâmico de dificuldade

Sistemas baseados em regras representam conhecimento na forma SE condição ENTÃO ação, avaliadas por um mecanismo de inferência. São a base dos sistemas especialistas e são muito usados em jogos quando se deseja comportamento controlável e explicável. Hunicke (2005) defende o **ajuste dinâmico de dificuldade** (DDA), em que o jogo observa o desempenho do jogador e ajusta parâmetros em tempo real para manter o desafio adequado; nesta proposta, o DDA é implementado por um pequeno conjunto de regras.

6 APLICAÇÃO AO PROJETO DE GAMES

6.1 Integração do agente ao jogo

O agente seria implementado como um novo módulo de servidor, o **RivalAgentManager**, colocado ao lado dos sistemas já existentes (gerenciador de pedidos, gerenciador de motos, validação anti-trapaça e gerenciador de incidentes). Rodar no servidor é importante porque ele participa da economia de pedidos: assim, nenhum jogador consegue manipular o comportamento do rival.

A malha de ruas do jogo já é gerada de forma procedural pela ferramenta de construção do mapa, que conhece todos os cruzamentos, trechos de rua e becos. Esses dados seriam exportados como um **grafo**: cada cruzamento vira um nó e cada trecho de rua (ou beco) vira uma aresta, com o comprimento como custo base. Os obstáculos já possuem dados estruturados (tipo, posição e eixo), que o agente usa para atualizar o custo das arestas. No cliente, o rival seria desenhado com o mesmo modelo 3D dos motoboys do trânsito, com piloto de cor diferente e uma etiqueta com o nome.

Para não pesar o servidor, a máquina de estados é atualizada 5 vezes por segundo, o movimento é interpolado a cada quadro e o A* só é executado em eventos (aceitar pedido, coletar, rua bloqueada ou obstáculo novo). Como cada bairro tem por volta de 300 cruzamentos, uma busca A* leva menos de um milissegundo. O número de rivais por servidor seria pequeno (por exemplo, até três) e diminuiria conforme entram jogadores reais.

6.2 Entradas utilizadas pelo agente

Quadro 4 – Entradas (percepções) do agente

Entrada	Origem no jogo	Para que é usada
Posição, velocidade e direção do rival	Simulação do próprio agente	Saber onde está no grafo e controlar a pilotagem
Mural de pedidos livres	Gerenciador de pedidos (restaurante, cliente, pagamento base, tempo ideal)	Árvore de decisão de avaliação de pedidos
Grafo de ruas e becos	Dados do gerador do mapa (cruzamentos, trechos, comprimentos)	Busca A* de rotas
Obstáculos no caminho	Dados dos perigos: espinhos, óleo, buracos, lombadas, lama, radares (com limite), obras e desvios	Custo das arestas, replanejamento e reações táticas

Entrada	Origem no jogo	Para que é usada
Trânsito	Posição dos veículos por trecho	Custo por congestionamento, frear e trocar de faixa
Pedestres	Posição dos pedestres nas calçadas	Evitar atropelamentos e decidir se corta pela calçada
Jogador	Posição, pedido atual e nível da moto	Disputa de pedidos e ajuste de dificuldade
Estado interno	Memória do agente (pneu furado, tempo sem progresso, histórico de disputas)	Transições de exceção e regras de dificuldade

Fonte: elaborado pela equipe (2026).

6.3 Decisões e ações produzidas pelo agente

Quadro 5 – Decisões e ações do agente

Decisão / ação	Quando acontece	Técnica responsável
Aceitar ou recusar pedido	Ao surgir um pedido livre no mural	Árvore de decisão
Definir destino e rota	Após aceitar e após coletar	A*
Replanejar rota	Rua bloqueada, obstáculo novo ou congestionamento	FSM + A*
Acelerar, frear e virar	A cada atualização, seguindo os pontos da rota	Controle de pilotagem (steering)
Trocar de faixa ou frear	Carro parado ou espinho ocupando só uma faixa	Árvore de decisão tática
Usar turbo ou saltar rampa	Reta longa sem obstáculos / rampa no caminho	Regras de pilotagem
Cortar pela calçada	Somente quando a utilidade é positiva e não há pedestres	Função de utilidade
Coletar e entregar	Ao entrar no raio de 16 studs do ponto	FSM
Ajustar agressividade e reação	Ao fim de cada disputa com o jogador	Sistema de regras (DDA)

Fonte: elaborado pela equipe (2026).

6.4 Interação entre o agente e os demais elementos do jogo

- **Sistema de pedidos:** o mural passa a ser compartilhado. Ao aceitar um pedido, o rival o reserva; se o jogador aceitar primeiro, o rival desiste. Pedidos parados por muito tempo podem ser “roubados” pelo rival, e o jogador é avisado.
- **Jogador:** a colisão entre o rival e o jogador apenas empurra, sem dano e sem taxa. Os dois disputam o placar da rodada, e o desempenho do jogador alimenta as regras de dificuldade.

- **Trânsito:** o rival obedece às mesmas regras do jogador, freando atrás dos carros e trocando de faixa. Os veículos do trânsito também freiam quando o rival está à frente.
- **Obstáculos:** causam no rival os mesmos efeitos que no jogador (pneu furado por 3 segundos, derrapagem no óleo, perda de velocidade na lama), o que torna a disputa justa e mostra ao jogador que evitar perigos compensa.
- **Pedestres e calçada:** o rival evita atropelamentos e só sobe na calçada quando o atalho compensa a lentidão e a taxa progressiva, sempre sem pedestres por perto.
- **Economia e servidor:** o rival não recebe dinheiro real (apenas pontuação), então não interfere no progresso do jogador. Como roda no servidor, respeita a validação anti-trapaça existente.

6.5 Ajuste dinâmico de dificuldade

Quadro 6 – Regras de ajuste dinâmico de dificuldade

Regra	Condição (SE)	Ajuste (ENTÃO)
R1	O jogador venceu 3 disputas seguidas	Agressividade +0,1 (máx. 0,8) e tempo de reação −0,2 s (mín. 0,5 s)
R2	O jogador perdeu 3 disputas seguidas	Agressividade −0,1 (mín. 0,2) e tempo de reação +0,3 s (máx. 2,5 s)
R3	A moto do jogador está no nível N	Velocidade do rival entre 90% e 105% da velocidade do nível N
R4	O jogador ainda está no tutorial (0 entregas)	Rivais desativados
R5	O jogador está atrasado (mais de 150% do tempo ideal)	O rival não disputa pedidos do mesmo restaurante, para evitar frustração
R6	Há jogadores reais suficientes no servidor	Número de rivais = máx(0, 3 − número de jogadores)

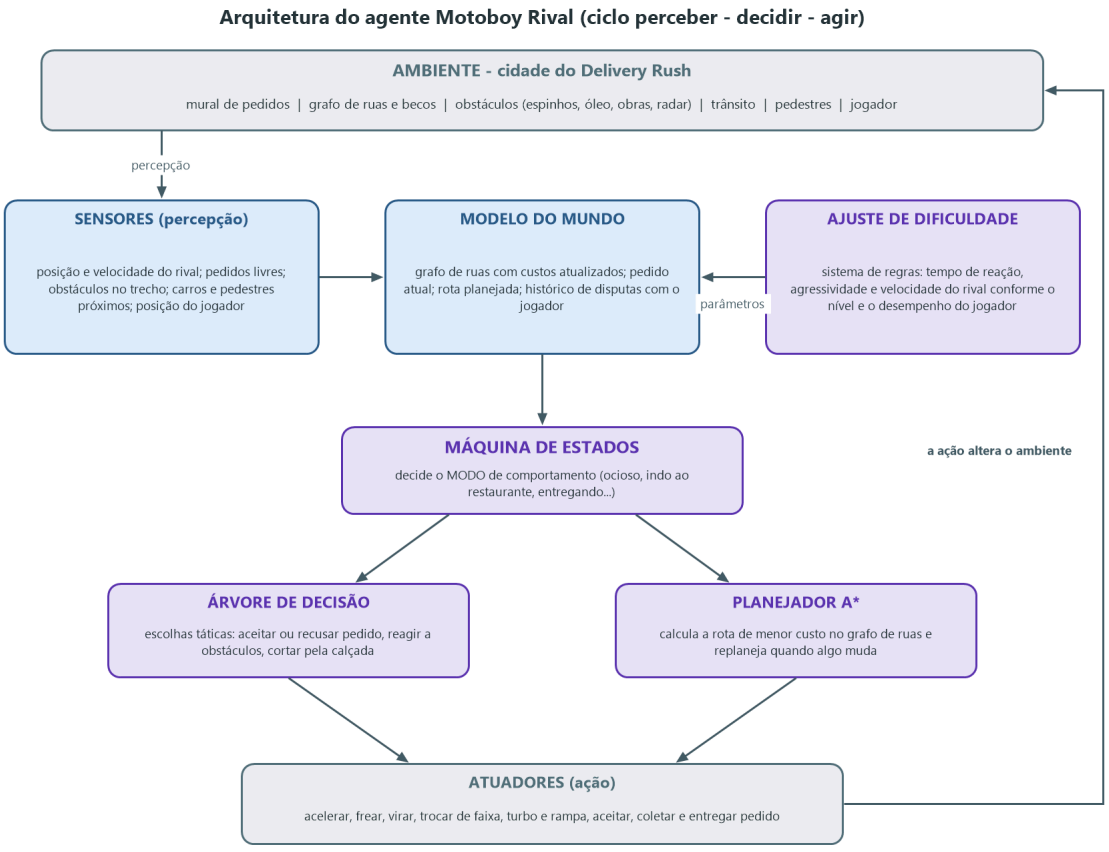
Fonte: elaborado pela equipe (2026).

7 MODELAGEM DA SOLUÇÃO

7.1 Arquitetura do agente

A Figura 1 mostra o ciclo perceber–decidir–agir do Motoboy Rival. Os sensores coletam informações do ambiente e atualizam o modelo do mundo. A máquina de estados decide o modo de comportamento e, dentro de cada estado, a árvore de decisão resolve as escolhas táticas enquanto o planejador A* calcula as rotas. As ações executadas alteram o ambiente, fechando o ciclo. As regras de ajuste de dificuldade regulam os parâmetros do agente.

Figura 1 – Arquitetura do agente Motoboy Rival



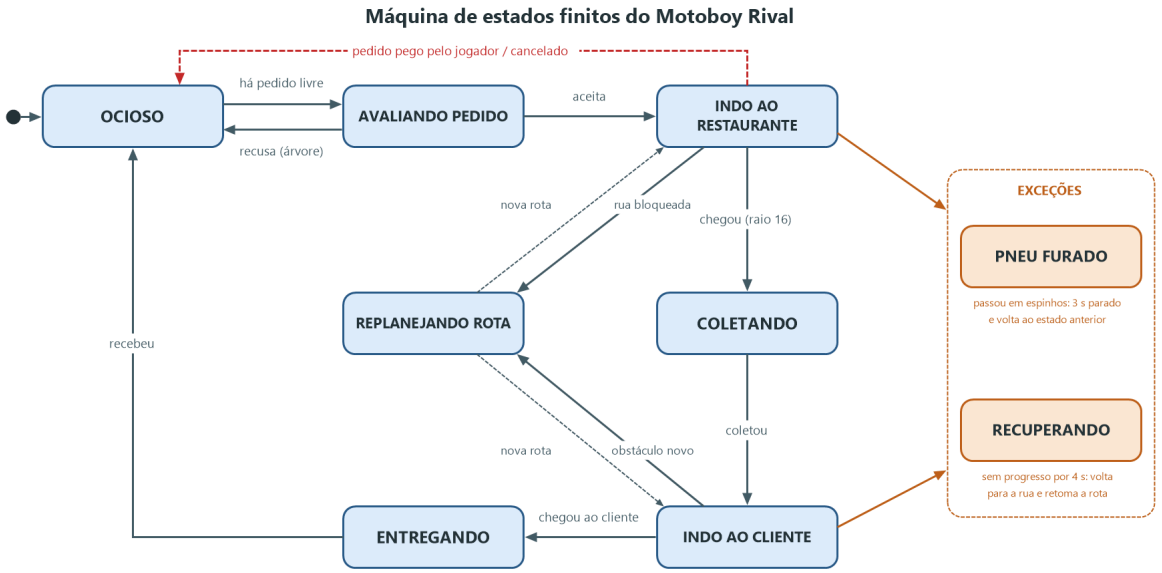
Fonte: elaborado pela equipe (2026).

7.2 Máquina de estados

A Figura 2 apresenta a máquina de estados do agente. O ciclo principal (ocioso → avaliando pedido → indo ao restaurante → coletando → indo ao cliente → entregando) representa uma entrega. O estado de replanejamento é acionado quando a rota deixa de ser válida, e os estados de exceção (pneu furado e

recuperando) podem ser alcançados a partir de qualquer estado de deslocamento; depois deles, o agente volta ao estado anterior.

Figura 2 – Máquina de estados finitos do Motoboy Rival



Fonte: elaborado pela equipe (2026).

Quadro 7 – Tabela de transições da máquina de estados

Estado atual	Evento / condição	Próximo estado	Ação ao entrar
OCIOSO	Há pedido livre no mural	AVALIANDO PEDIDO	Consultar a árvore de decisão
AVALIANDO PEDIDO	Árvore decide aceitar	INDO AO RESTAURANTE	Reservar pedido e calcular rota A*
AVALIANDO PEDIDO	Árvore decide recusar	OCIOSO	Circular até o centro do bairro
INDO AO RESTAURANTE	Chegou ao raio de 16 studs	COLETANDO	Parar e aguardar 1 s
INDO AO RESTAURANTE	Jogador pegou o pedido / pedido cancelado	OCIOSO	Liberar reserva
COLETANDO	Pedido coletado	INDO AO CLIENTE	Calcular rota A* até o cliente
INDO AO CLIENTE	Chegou ao raio de 16 studs	ENTREGANDO	Parar e registrar a entrega
ENTREGANDO	Entrega registrada	OCIOSO	Somar pontos e atualizar o histórico
Indo ao restaurante / cliente	Rua bloqueada ou obstáculo novo	REPLANEJANDO ROTA	Atualizar custos e executar A*

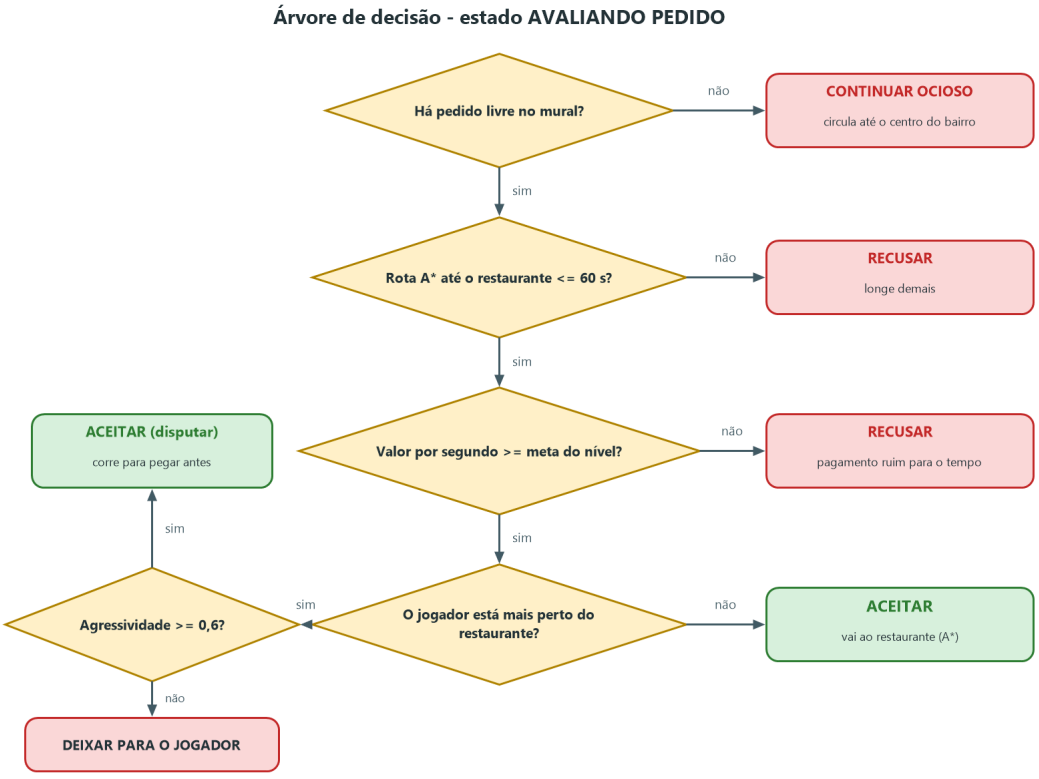
Estado atual	Evento / condição	Próximo estado	Ação ao entrar
REPLANEJANDO ROTA	Nova rota calculada	Estado anterior	Seguir a nova rota
Indo ao restaurante / cliente	Passou em espinhos	PNEU FURADO	Parar por 3 s
Indo ao restaurante / cliente	Sem progresso por 4 s	RECUPERANDO	Voltar ao trecho de rua mais próximo

Fonte: elaborado pela equipe (2026).

7.3 Árvore de decisão para avaliação de pedidos

No estado AVALIANDO PEDIDO, o agente percorre a árvore da Figura 3. O atributo **valor por segundo** é o pagamento estimado dividido pelo tempo total previsto (tempo até o restaurante + tempo do restaurante até o cliente, ambos calculados pelo A*). A **meta do nível** é o valor mínimo por segundo que o rival aceita e cresce conforme o bairro. A **agressividade**, entre 0 e 1, é controlada pelas regras de dificuldade e define se o rival disputa pedidos que o jogador tem mais chance de pegar.

Figura 3 – Árvore de decisão do estado AVALIANDO PEDIDO



Fonte: elaborado pela equipe (2026).

7.4 Planejamento de rotas com A*

O custo de cada aresta do grafo é o **tempo esperado** para percorrê-la, somado às penalidades dos obstáculos. Penalidades em dinheiro são convertidas em segundos pelo valor por segundo atual do agente (por exemplo, 2 $/s):

custo(e) = (comprimento / velocidade) × fator de trânsito + Σ penalidades dos obstáculos

A heurística é h(n) = distância em linha reta até o objetivo ÷ velocidade máxima do rival (incluindo o turbo). Ela é admissível, pois nenhum caminho real pode ser mais rápido do que seguir em linha reta na velocidade máxima; isso garante que o A* encontre a rota ótima.

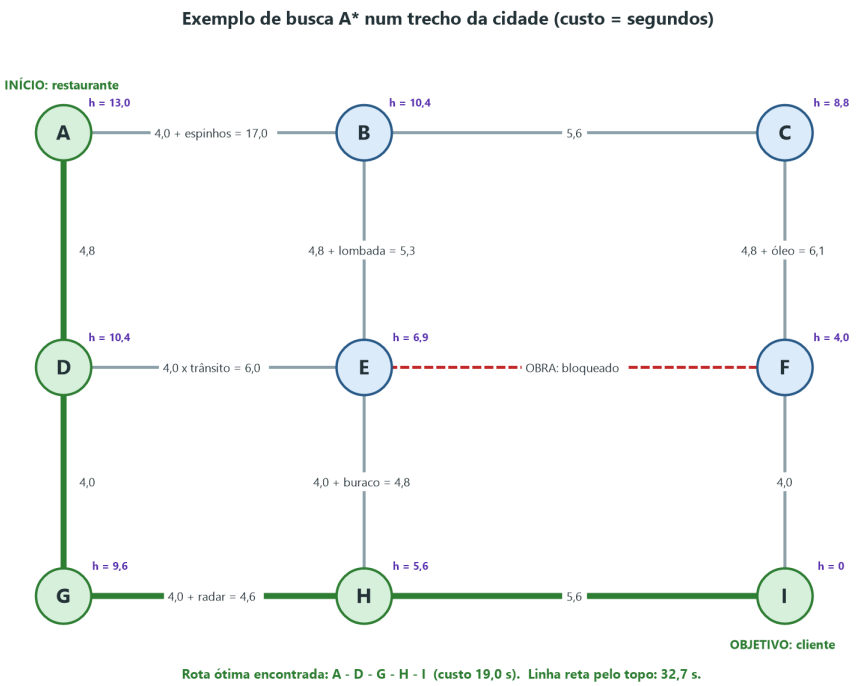
Quadro 8 – Penalidades de custo por obstáculo

Obstáculo	Efeito no jogo	Penalidade na aresta
Espinhos	3 s parado + $20 de conserto	+3 s + 20 ÷ (valor por segundo) ≈ +13 s
Óleo	Derrapagem de 1,3 s	+1,3 s
Buraco	Tranco e perda de 40% da velocidade	+0,8 s
Lombada	Perde 30% da velocidade se passar rápido	+0,5 s
Lama	Perde 55% da velocidade	+2,0 s
Hidrante (água)	Escorregada de 0,8 s	+0,8 s
Radar	O rival reduz a velocidade para não ser multado	+0,6 s
Trânsito	Veículos lentos na faixa	multiplica o tempo por 1,0 a 1,8
Desvio	Uma faixa fechada	+1,5 s
Obra	Rua totalmente fechada	∞ (aresta bloqueada)

Fonte: elaborado pela equipe (2026).

A Figura 4 mostra um exemplo em um trecho da cidade com nove cruzamentos (A a I). O pedido deve ser levado do restaurante, em A, até o cliente, em I. Os custos das arestas estão em segundos (velocidade de referência de 40 studs/s) e os valores de h aparecem ao lado de cada nó.

Figura 4 – Exemplo de busca A* em um trecho da cidade



Fonte: elaborado pela equipe (2026).

Quadro 9 – Execução passo a passo do A* no exemplo da Figura 4

Passo	Nó expandido	Lista aberta após a expansão (nó: g / f)	Observação
1	A (g = 0; f = 13,0)	D: 4,8 / 15,2 — B: 17,0 / 27,4	A–B tem espinhos e fica caro
2	D	E: 10,8 / 17,7 — G: 8,8 / 18,4 — B: 17,0 / 27,4	
3	E	G: 8,8 / 18,4 — H: 15,6 / 21,2 — B: 16,1 / 26,5	E–F ignorada (obra, custo ∞); B melhora via E
4	G	H: 13,4 / 19,0 — B: 16,1 / 26,5	H melhora via G (evita o buraco)
5	H	I: 19,0 / 19,0 — B: 16,1 / 26,5	
6	I	—	Objetivo alcançado: A–D–G–H–I, custo 19,0 s

Fonte: elaborado pela equipe (2026).

O A* expandiu apenas 6 dos 9 nós (B, C e F nunca foram expandidos) e encontrou uma rota de 19,0 segundos, que evita os espinhos, a obra e o congestionamento. A rota “ingênua” pelo topo do mapa (A–B–C–F–I), que parece a mais direta, custaria 32,7 segundos, ou seja, 72% a mais.

7.5 Decisão de cortar caminho pela calçada

A calçada não faz parte do grafo, pois andar nela deixa a moto mais lenta e gera uma taxa que cresce a cada 1,6 segundo ($3, $4, $5...). Por isso, cortar caminho é uma decisão pontual, tomada por uma **função de utilidade**:

U = (tempo economizado × valor por segundo) − taxas previstas − risco de atropelamento

O risco é estimado como (número de pedestres no trecho × probabilidade de colisão × multa de $50). O agente só corta caminho se U > 0 e se a agressividade for de pelo menos 0,5. Exemplo: um atalho economiza 7 s e o valor por segundo é 3 $/s, o que dá um ganho de $21. O trecho leva 3,2 s na calçada (2 cobranças: $3 + $4 = $7). Com 2 pedestres no caminho e probabilidade de colisão de 0,3, o risco é 2 × 0,3 × 50 = $30, logo U = 21 − 7 − 30 = −16 e o agente **não corta**. Sem pedestres, U = 21 − 7 = 14 e o agente **corta**.

7.6 Pseudocódigo do agente

```
função atualizar(rival)                       -- executada 5 vezes por segundo
    percepção ← perceber(rival)                -- sensores
    rival.modelo.atualizar(percepção)
    se percepção.pneuFurado então mudarEstado(PNEU_FURADO)
    se rival.tempoSemProgresso > 4 então mudarEstado(RECUPERANDO)
    escolha rival.estado
      OCIOSO:
        se existe pedido livre então mudarEstado(AVALIANDO_PEDIDO)
        senão circular()
      AVALIANDO_PEDIDO:
        pedido ← arvoreAvaliarPedido(percepção)
        se pedido ≠ nulo então
          reservar(pedido); rota ← A*(posição, pedido.restaurante)
          mudarEstado(INDO_AO_RESTAURANTE)
        senão mudarEstado(OCIOSO)
      INDO_AO_RESTAURANTE, INDO_AO_CLIENTE:
        se pedido cancelado então mudarEstado(OCIOSO)
        senão se rotaInvalida(rota, percepção) então mudarEstado(REPLANEJANDO_ROTA)
        senão seguirRota(rota, percepção)     -- pilotagem + reações táticas
      REPLANEJANDO_ROTA:
        rota ← A*(posição, destinoAtual); voltarAoEstadoAnterior()

função A*(início, objetivo)
    aberta ← {início}; g[início] ← 0; f[início] ← h(início)
    enquanto aberta não estiver vazia
        n ← nó de aberta com menor f
        se n = objetivo então retornar reconstruirCaminho(n)
        mover n de aberta para fechada
        para cada aresta e de n até m
            se custo(e) = ∞ ou m ∈ fechada então continuar
            gNovo ← g[n] + custo(e)
            se m ∉ aberta ou gNovo < g[m] então
                pai[m] ← n; g[m] ← gNovo; f[m] ← gNovo + h(m)
                colocar m em aberta
    retornar falha
```

7.7 Exemplos de situações de entrada e decisões esperadas

Quadro 10 – Situações de entrada e decisões esperadas do agente

Situação (entradas)	Estado	Decisão esperada	Técnica
Pedido livre: restaurante a 14 s pela rota A*, pagamento $180, tempo total 48 s (3,75 $/s ≥ meta 2,5); jogador longe	AVALIANDO PEDIDO	Aceitar e ir ao restaurante	Árvore + A*
Pedido a 58 s, pagamento $150, tempo total 95 s (1,6 $/s < meta)	AVALIANDO PEDIDO	Recusar e continuar ocioso	Árvore
Jogador a 200 studs do restaurante e rival a 500; agressividade 0,3	AVALIANDO PEDIDO	Deixar o pedido para o jogador	Árvore + DDA
Mesmo caso anterior, mas o jogador venceu 3 disputas seguidas (agressividade 0,7)	AVALIANDO PEDIDO	Aceitar e disputar o pedido	Árvore + DDA
Obra surge em uma rua da rota planejada	INDO AO CLIENTE	Replanejar com a aresta bloqueada (custo ∞) e seguir a nova rota	FSM + A*

Situação (entradas)	Estado	Decisão esperada	Técnica
Espinhos ocupando só a faixa da direita, 40 studs à frente	INDO AO RESTAURANTE	Trocar para a faixa livre sem mudar a rota	Árvore tática
Passou em espinhos que ocupavam as duas faixas	INDO AO CLIENTE	Pneu furado: parar 3 s e retomar a rota	FSM
Carro parado 12 studs à frente, faixa ao lado ocupada	INDO AO CLIENTE	Frear e aguardar a faixa liberar	Árvore tática
Radar de 90 km/h a 80 studs; rival a 110 km/h	INDO AO CLIENTE	Reduzir para 88 km/h até passar o radar	Regras de pilotagem
Atalho pela calçada economiza 7 s, com 2 pedestres no trecho	INDO AO CLIENTE	Não cortar (utilidade negativa)	Função de utilidade
Rival parado há mais de 4 s, encostado em uma parede	INDO AO RESTAURANTE	Recuperar: voltar ao trecho de rua mais próximo e replanejar	FSM + A*
O jogador aceita o pedido que o rival estava indo buscar	INDO AO RESTAURANTE	Cancelar e voltar a ficar ocioso	FSM

Fonte: elaborado pela equipe (2026).

8 CONSIDERAÇÕES FINAIS

Este relatório propôs o Motoboy Rival, um agente inteligente que transforma os NPCs de Delivery Rush de simples elementos decorativos em competidores que participam da economia de entregas. A combinação de uma máquina de estados (comportamento), uma árvore de decisão (escolhas táticas), a busca A* (navegação) e um sistema de regras (ajuste dinâmico de dificuldade) atende aos requisitos levantados: o rival escolhe pedidos rentáveis, encontra rotas ótimas que evitam obstáculos, reage a mudanças no ambiente, respeita as regras do jogo e mantém a disputa equilibrada.

Entre os benefícios esperados estão a sensação de uma cidade viva, a pressão de tempo natural na escolha dos pedidos, mais variedade entre as partidas e servidores com pouca gente que não parecem vazios. As técnicas escolhidas são leves, interpretáveis e se encaixam na arquitetura cliente–servidor que o jogo já utiliza.

Como trabalhos futuros, sugere-se: (a) usar **aprendizado por reforço** (SUTTON; BARTO, 2018) em simulações fora do jogo para ajustar automaticamente os parâmetros do agente (metas de valor por segundo, penalidades e agressividade); (b) gerar a árvore de decisão com o algoritmo **ID3**, a partir de registros de partidas de bons jogadores; e (c) criar “personalidades” de rivais (cauteloso, arrojado e trapalhão) variando esses parâmetros.

REFERÊNCIAS

BUCKLAND, M. **Programming game AI by example**. Plano: Wordware Publishing, 2005.

CSIKSZENTMIHALYI, M. **Flow**: the psychology of optimal experience. New York: Harper & Row, 1990.

HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. **IEEE Transactions on Systems Science and Cybernetics**, v. 4, n. 2, p. 100-107, 1968.

HUNICKE, R. The case for dynamic difficulty adjustment in games. In: ACM SIGCHI INTERNATIONAL CONFERENCE ON ADVANCES IN COMPUTER ENTERTAINMENT TECHNOLOGY, 2005, Valencia. **Proceedings** [...]. New York: ACM, 2005. p. 429-433.

MILLINGTON, I.; FUNGE, J. **Artificial intelligence for games**. 2. ed. Burlington: Morgan Kaufmann, 2009.

QUINLAN, J. R. Induction of decision trees. **Machine Learning**, v. 1, n. 1, p. 81-106, 1986.

ROBLOX CORPORATION. **Roblox Creator Documentation**. Disponível em: https://create.roblox.com/docs. Acesso em: 25 set. 2026.

RUSSELL, S.; NORVIG, P. **Inteligência artificial**: uma abordagem moderna. 4. ed. Rio de Janeiro: GEN LTC, 2022.

SUTTON, R. S.; BARTO, A. G. **Reinforcement learning**: an introduction. 2. ed. Cambridge: MIT Press, 2018.

YANNAKAKIS, G. N.; TOGELIUS, J. **Artificial intelligence and games**. Cham: Springer, 2018.